

Laboratorio 1: Programación paralela con OpenMP

Simulador gravitatorio N-cuerpos en 2D (C++)

Departamento de Ingeniería Informática
Universidad de Santiago de Chile
Prof. Miguel Cárcamo
Ayudante: Diego Fernández

2 de abril de 2026

1. Objetivos

El laboratorio integra física newtoniana elemental, diseño orientado a objetos en C++ y análisis de rendimiento con OpenMP. Al finalizar, ustedes serán capaces de:

- Implementar aplicaciones paralelas usando OpenMP
- Utilizar diferentes cláusulas de scheduling en OpenMP
- Aplicar cláusulas de sincronización (atomic, critical, reduce)
- Calcular métricas de rendimiento (speedup, eficiencia)
- Analizar la ley de Amdahl en aplicaciones paralelas
- Generar gráficos y tablas de resultados experimentales
- Desarrollar código orientado a objetos en C++
- Implementar clases y métodos con OpenMP
- Implementar gravitación newtoniana pareja-a-pareja con suavizado (sin rejilla espacial)
- Discutir reproducibilidad numérica, estabilidad del integrador y límites del modelo $\mathcal{O}(N^2)$
- Diseñar y ejecutar pruebas unitarias y de integración sobre el simulador

2. Organización del equipo (cinco integrantes)

Cada grupo tiene cinco integrantes. Cada persona asume un rol principal y puede colaborar en el resto. En el `README.md` y en el reporte incluyan una tabla con nombre, rol y responsabilidades concretas. En el test de salida del curso se les podrá preguntar por el trabajo asociado a su rol.

Rol	Responsabilidades típicas
1. Modelo y datos	<code>Particle</code> , contenedor del sistema: masas, posiciones, velocidades, constante G , suavizado ε , inicialización reproducible (semilla), I/O de estados y <code>.dat</code> .
2. Núcleo paralelo	<code>NBodySystem::computeAccelerations</code> (o equivalente): bucles pareja-a-pareja o “todo-pares”, OpenMP, schedules, <code>collapse</code> donde aplique, ausencia de condiciones de carrera al escribir aceleraciones por cuerpo.
3. Integración y física	<code>Integrator</code> / <code>NBodySimulator</code> : Euler (mínimo) u opcionalmente Verlet/ leapfrog documentado; paso Δt , criterios de estabilidad y conservación aproximada de energía.
4. Métricas y benchmarks	<code>MetricsCalculator</code> , <code>Benchmark</code> : mediciones con <code>omp_get_wtime()</code> , repeticiones, estadística, speedup, eficiencia, Amdahl, propagación de errores, scripts de experimentos.
5. Calidad, CI y visualización	Pruebas unitarias e integración (p.ej. GoogleTest o Catch2), contenedor Docker, pipeline CI, <code>Visualizer</code> o scripts Python/Gnuplot, integración del objetivo <code>make test</code> .

3. Calendario sugerido

El enunciado se publica el 2 de abril de 2026 y la entrega es el 8 de mayo de 2026, es decir, alrededor de cinco semanas. La siguiente división es orientativa; ajusten los hitos en el README según su ritmo.

- Semana 1: repositorio, Docker y CI mínimo; `Particle` y sistema serial; fuerza entre dos cuerpos y caso $N = 3$ verificado a mano; primeras pruebas unitarias de distancia y aceleración.
- Semana 2: `computeAccelerations` en paralelo; comparación serial frente a paralelo con N pequeño y tolerancia en coma flotante justificada.
- Semana 3: `NBodySimulator` y `Integrator`; lazo temporal; energía cinética y potencial; pruebas de integración en pocos pasos.
- Semana 4: `Benchmark`, experimentos, gráficos y tablas con incertidumbre; registro de máquina y flags de compilación.
- Semana 5: CI estable con `make test`, contenedor pulido, reporte en PDF y revisión cruzada por rol.

4. Descripción del problema

Se pide un simulador gravitatorio de N cuerpos en el plano: cada partícula tiene masa, posición (x_i, y_i) y velocidad. Las interacciones siguen la ley de gravitación newtoniana entre todos los pares, con un suavizado tipo Plummer $\varepsilon > 0$ que evita la singularidad cuando dos posiciones coinciden. El costo del método directo es $\mathcal{O}(N^2)$ por paso temporal. No se solicitan algoritmos de tipo Barnes–Hut ni PM/PP; el objetivo pedagógico es dominar el patrón todo-pares con OpenMP, medir rendimiento y razonar sobre los resultados. La solución debe estar organizada en clases en C++.

4.1. Contexto

Las integraciones N-cuerpos aparecen en astrofísica (cúmulos, anillos, sistemas estelares globulares, entre otros). Trabajar en 2D simplifica la visualización y el código, pero mantiene el reto de fuerzas de largo alcance y de acumular muchas interacciones por paso.

En resumen, el modelo supone lo siguiente:

1. Cada cuerpo i tiene masa m_i , posición $\vec{r}_i = (x_i, y_i)$ y velocidad $\vec{v}_i = (v_{x,i}, v_{y,i})$.
2. La aceleración sobre i por efecto de $j \neq i$ es newtoniana, con distancia suavizada $s_{ij} = \sqrt{\|\vec{r}_j - \vec{r}_i\|^2 + \varepsilon^2}$.
3. En cada paso se calculan todas las aceleraciones a partir de las posiciones ya conocidas de ese instante; después se actualizan velocidades y posiciones con un integrador explícito (Euler como mínimo, ver más abajo).
4. Las condiciones iniciales deben poder regenerarse a partir de una semilla documentada, de modo que un tercero pueda repetir la corrida.

La aceleración del cuerpo i es

$$\vec{a}_i = G \sum_{j \neq i} m_j \frac{\vec{r}_j - \vec{r}_i}{(\|\vec{r}_j - \vec{r}_i\|^2 + \varepsilon^2)^{3/2}}. \quad (1)$$

Integración temporal mínima (Euler explícito). Una vez conocidas todas las $\vec{a}_i$,

$$\vec{v}_i \leftarrow \vec{v}_i + \vec{a}_i \Delta t, \quad (2)$$

$$\vec{r}_i \leftarrow \vec{r}_i + \vec{v}_i \Delta t. \quad (3)$$

Aquí G es la constante gravitacional en las unidades que elijan (por ejemplo $G = 1$ en un sistema adimensional), ε es el suavizado y Δt el paso temporal. En el reporte deben argumentar si la elección de Δt y ε mantiene el sistema numéricamente estable (con experimentos o estimaciones de orden de magnitud).

4.2. Algoritmo de simulación

Inicialización.

1. Definir N , G , ε , Δt y la semilla para posiciones o masas (por ejemplo disco aleatorio, perfil tipo Plummer en 2D, o dos cuerpos con perturbación).
2. Asignar $\vec{r}_i$, $\vec{v}_i$, m_i .

Bucle temporal.

1. Anular y volver a calcular $\vec{a}_i$ sumando la contribución de todo $j \neq i$.
2. Aplicar el integrador sobre $\vec{v}_i$ y $\vec{r}_i$ usando solo los valores de ese paso (primero todas las aceleraciones; después kick y drift, sin entremezclar).
3. Opcional: guardar energía, momento o trayectorias para el análisis.

4.2.1. Convención de bucles (referencia del curso)

Para alinear implementación, pruebas y corrección, se adopta el siguiente esquema salvo que documenten otra variante equivalente en el reporte:

- El cálculo de aceleraciones usa un bucle paralelo sobre el índice de cuerpo i ; cada iteración escribe únicamente $\vec{a}_i$. El bucle interno sobre $j \neq i$ corre en serial dentro del hilo. Así se evitan carreras sin duplicar pares (i, j) y (j, i) .
- El método `computeAccelerationsCollapse()` aplica solo si implementan un doble bucle explícito sobre índices y usan `collapse` de forma válida; deben demostrar equivalencia con la ecuación de $\vec{a}_i$.
- Orden fijo del Euler en cada paso: (1) todas las $\vec{a}_i$; (2) todas las $\vec{v}_i \leftarrow \vec{v}_i + \vec{a}_i \Delta t$; (3) todas las $\vec{r}_i \leftarrow \vec{r}_i + \vec{v}_i \Delta t$. Si experimentan otro orden, documenten y comparen.
- Si fijan $G = 1$, declaren en el README un sistema adimensional coherente (masas, longitudes, tiempos) para todo el repositorio.

Ejemplo numérico. Dos masas en el eje x : m_1 en $x = 0$, m_2 en $x = d > 0$. Sobre la partícula 1, la aceleración (solo componente x) es $a_{1,x} = Gm_2 \frac{d}{(d^2 + \varepsilon^2)^{3/2}}$ (dirigida hacia $+x$). Con $G = m_2 = d = 1$, $\varepsilon = 0,1$: $a_{1,x} = 1/(1 + 0,01)^{3/2} \approx 0,971$.

4.3. Fundamentos teóricos

4.3.1. Energía y cantidades conservadas

Energía cinética:

$$K = \frac{1}{2} \sum_i m_i \|\vec{v}_i\|^2. \quad (4)$$

Energía potencial (suma sobre pares distintos, con el mismo suavizado):

$$U = -G \sum_{i < j} \frac{m_i m_j}{\sqrt{\|\vec{r}_j - \vec{r}_i\|^2 + \varepsilon^2}}. \quad (5)$$

Con Euler explícito, $E = K + U$ no se conserva de manera exacta; conviene mostrar en el reporte cómo deriva la energía al variar Δt y cómo el paralelismo puede alterar el orden de sumas en las métricas. El momento lineal total debería mantenerse aproximadamente si solo hay fuerzas internas y el par (i, j) se trata de forma simétrica; mídanlo y comenten cualquier desviación sistemática.

4.3.2. Paralelismo y reproducibilidad

El esquema recomendado paraleliza el bucle sobre i : cada hilo escribe solo $\vec{a}_i$ y lee posiciones compartidas. Para K , U u otras sumas globales suelen usarse `reduce`, `atomic` o acumuladores locales con una reducción final; lo importante es documentar la estrategia y justificar su corrección.

Comparen siempre una versión serial de referencia frente a la paralela. Incluso con la misma lógica, en coma flotante pueden aparecer diferencias pequeñas; definan en los tests una tolerancia razonable y expliquen cómo la eligieron. Si lo desean, mantengan un modo de un solo hilo exclusivamente para regresión.

4.3.3. Métricas sugeridas

Además de K y U definidos arriba, conviene reportar (con las mismas convenciones de suavizado donde aplique):

- **Momento lineal total.** $\vec{P} = \sum_i m_i \vec{v}_i$; su magnitud es $\|\vec{P}\|$.
- **Energía total.** $E(t) = K(t) + U(t)$.
- **Centro de masas.** $\vec{R}_{\text{cm}} = \frac{\sum_i m_i \vec{r}_i}{\sum_i m_i}$, con $M = \sum_i m_i$.
- **Radio del sistema (RMS respecto del centro de masas).** $R_{\text{rms}} = \sqrt{\frac{1}{M} \sum_i m_i \|\vec{r}_i - \vec{R}_{\text{cm}}\|^2}$.
- **Distancia mínima entre pares.** $d_{\text{mín}} = \min_{i < j} \|\vec{r}_j - \vec{r}_i\|$ (útil para detectar encuentros muy cercanos).

5. Requisitos técnicos

5.1. Introducción a OpenMP

OpenMP (Open Multi-Processing) es una interfaz para paralelizar C/C++ mediante directivas del compilador. Las directivas comienzan con `#pragma omp`.

5.2. Cláusulas OpenMP obligatorias

La implementación debe cubrir, al menos una vez de forma correcta y comprobable, las siguientes familias de cláusulas:

1. *Scheduling* (reparto de iteraciones):

- `schedule(static, chunk)` — reparto estático con tamaño de *chunk* fijo
- `schedule(dynamic, chunk)` — reparto dinámico con *chunk* fijo
- `schedule(guided, chunk)` — reparto guiado con tamaño mínimo de *chunk*

2. *Sincronización básica* sobre variables compartidas:

- `atomic` — operaciones atómicas (suele ser más barato que `critical` cuando aplica)
- `critical` — sección crítica genérica
- `reduce` — reducción paralela (suma, producto, etc.)

3. *Otras directivas frecuentes*:

- `single` — un solo hilo ejecuta el bloque
- `nowait` — omite la barrera implícita al final de la región
- `collapse` — fusiona bucles anidados en un solo espacio de iteraciones

4. *Manejo de datos*:

- `private, shared`

- `firstprivate`, `lastprivate`

5. Sincronización y tareas:

- `barrier`
- `task`

5.3. Análisis de rendimiento

Se espera un módulo de benchmarks coherente con el simulador. Debe contemplar lo siguiente:

1. Medición de tiempos con `omp_get_wtime()`, repitiendo cada experimento al menos diez veces, reportando promedio y desviación estándar en la forma $T = \bar{T} \pm \sigma_T$.

2. Speedup

$$S_p = \frac{T_1}{T_p} \quad (6)$$

donde T_1 es el tiempo con un solo hilo OpenMP (`OMP_NUM_THREADS=1` o equivalente), mismo binario y misma entrada que el caso con p hilos (T_p), y mismos flags de compilación.

3. Eficiencia

$$E_p = \frac{S_p}{p} \quad (7)$$

con $0 \leq E_p \leq 1$ en el caso ideal.

4. Ley de Amdahl, en la forma usual

$$S_p = \frac{1}{f + \frac{1-f}{p}} \quad (8)$$

donde f es la fracción estrictamente serial del programa ($0 \leq f \leq 1$), p el número de hilos y la fórmula acota el speedup teórico. En la práctica pueden estimar f a partir de tiempos instrumentados.

5. Gráficos de rendimiento (Python/Matplotlib, Gnuplot u otra herramienta equivalente).

5.3.1. Propagación de errores

Si speedup y eficiencia se calculan a partir de tiempos con incertidumbre, deben propagar esos errores en el reporte (al menos donde presenten barras de error en S_p o E_p).

Si $z = f(x, y)$ y x, y tienen errores σ_x y σ_y , una estimación habitual es

$$\sigma_z = \sqrt{\left(\frac{\partial f}{\partial x} \sigma_x\right)^2 + \left(\frac{\partial f}{\partial y} \sigma_y\right)^2}. \quad (9)$$

Para $S_p = T_1/T_p$,

$$\sigma_{S_p} = S_p \sqrt{\left(\frac{\sigma_{T_1}}{T_1}\right)^2 + \left(\frac{\sigma_{T_p}}{T_p}\right)^2}. \quad (10)$$

Para $E_p = S_p/p$, con p fijo,

$$\sigma_{E_p} = \frac{\sigma_{S_p}}{p}. \quad (11)$$

Ejemplo numérico. Con $T_1 = 9,97 \pm 0,15$ s y $T_4 = 3,01 \pm 0,12$ s:

$$\sigma_{S_4} = 3,31 \sqrt{\left(\frac{0,15}{9,97}\right)^2 + \left(\frac{0,12}{3,01}\right)^2} = 0,14 \quad (12)$$

$$\sigma_{E_4} = \frac{0,14}{4} = 0,035 \quad (13)$$

Resultado: $S_4 = 3,31 \pm 0,14$, $E_4 = 0,83 \pm 0,04$

6. Estructura del proyecto

6.1. Archivos requeridos

Se sugiere la siguiente organización (pueden añadir archivos auxiliares si lo justifican):

```
nbody_2d/
|-- main.cpp           # Archivo principal
|-- Particle.h/.cpp    # Masa, posición, velocidad, aceleración (2D)
|-- NBodySystem.h/.cpp # Conjunto de partículas, G, epsilon; computeAccelerations*
|-- NBodySimulator.h/.cpp # Lazo temporal, variantes OpenMP (orquestación)
|-- Integrator.h/.cpp  # Paso de Euler (u otros documentados)
|-- MetricsCalculator.h/.cpp # K, U, momento, etc.
|-- Benchmark.h/.cpp
|-- Visualizer.h/.cpp  # Trayectorias / snapshots para graficar
|-- tests/
|-- Dockerfile
|-- Makefile
`-- README.md
```

6.2. Programación orientada a objetos

- Clases en pares `.h/.cpp` cuando corresponda
- Métodos que concentren el uso de cada cláusula OpenMP requerida (vía sobrecarga, según la siguiente sección)
- Encapsulación clara de datos y operaciones
- Comentarios de clase o breve documentación en el README
- Herencia opcional (por ejemplo distintas condiciones iniciales: disco, perfil Plummer 2D, sistema binario)

7. Especificaciones de implementación

7.1. Diseño de clases

Las responsabilidades esperadas son las siguientes (los nombres pueden cambiar si el diseño queda explicado en el README):

- `Particle`: estado 2D del cuerpo y acceso controlado

- `NBodySystem`: contenedor, G , ε , cálculo de aceleraciones con variantes OpenMP
- `NBodySimulator`: lazo temporal, `integrateEuler`, métricas y rutas instrumentadas
- `Integrator`: actualización explícita de $\vec{v}$ y $\vec{r}$; puede fusionarse con `NBodySimulator` si separan responsabilidades por escrito
- `MetricsCalculator`: K , U , momento, reducciones
- `Benchmark`: tiempos de `computeAccelerations` y del paso completo frente a N y número de hilos
- `Visualizer`: exportación de trayectorias o instantáneas

7.1.1. Ejemplos ilustrativos de clases

Particle.

```

1  class Particle {
2  private:
3      double mass;
4      double x, y, vx, vy, ax, ay;
5  public:
6      Particle(double m, double x0, double y0);
7      void setAcceleration(double ax_, double ay_);
8      void addAcceleration(double dax, double day);
9      void kick(double dt);    // v += a*dt
10     void drift(double dt);   // r += v*dt
11     double getMass() const;
12     // getters/setters de estado...
13 };

```

NBodySystem.

```

1  class NBodySystem {
2  private:
3      std::vector<Particle> bodies;
4      double G_const;
5      double softening_eps;
6  public:
7      NBodySystem(double G, double epsilon);
8      void addParticle(const Particle& p);
9      void zeroAccelerations();
10
11     // Sobrecarga: cálculo de aceleraciones con distintos schedules / collapse
12     void computeAccelerations();
13     void computeAccelerations(int schedule_type);
14     void computeAccelerations(int schedule_type, int chunk_size);
15     void computeAccelerationsCollapse(); // p.ej. collapse(2) en i,j
16

```



```

17     const std::vector<Particle>& getBodies() const;
18     int getCount() const;
19 };

```

NBodySimulator.

```

1  class NBodySimulator {
2  private:
3      NBodySystem* system;
4      double time_step;
5  public:
6      NBodySimulator(NBodySystem* sys, double dt);
7
8      void integrateEuler();
9      void integrateEuler(int sync_type);          // 0=atomic, 1=critical, 2=nowait
10     ↪ (convención del enunciado)
11     void integrateEuler(int sync_type, bool use_barrier);
12
13     void calculateEnergy();
14     void calculateEnergy(int method);            // reduce=0, atomic=1
15     void calculateEnergy(int method, bool use_private);
16
17     void processBodies();
18     void processBodies(int task_type);           // task=0, parallel_for=1
19     void processBodies(int task_type, bool use_single);
20
21     void simulatePhasesBarrier();
22     void parallelInitializationSingle();
23 };

```

7.2. Métodos obligatorios y sobrecarga (*function overloading*)

Sobrecarga frente a la física del integrador. Varias cláusulas OpenMP (`atomic`, `critical`, `reduce`, `nowait`, etc.) encajan de forma natural en el cálculo de energía o en fases separadas del paso (kick/drift) cuando se instrumenta el código con ese fin. No se pide que cada línea del Euler use todas las cláusulas; sí que cada ítem de la lista obligatoria aparezca en alguna ruta real (compilada y ejecutada en tests o benchmarks) y que el reporte cite archivo, función y razón de uso.

Las variantes `processBodies*` sirven para contrastar `task` y `parallel for` repartiendo bloques de índices i ; no reemplazan la física de `computeAccelerations*`, que debe implementar la suma sobre $j \neq i$ de la ecuación dada arriba.

`calculateMetricsFirstprivate()` y `calculateFinalStateLastprivate()` deben usar `firstprivate` y `lastprivate` en un contexto con sentido (por ejemplo acumuladores por hilo o variables que sobreviven al `for` paralelo). Pueden vivir en `NBodySimulator` o en `MetricsCalculator`; indíquenlo en el README.

1. `NBodySystem` — aceleraciones (todo-pares):

- `computeAccelerations()`

- `computeAccelerations(int schedule_type)` con 0=estático, 1=dinámico, 2=guiado (sobre el bucle paralelo en *i*, salvo variante documentada)
- `computeAccelerations(int schedule_type, int chunk.size)`
- `computeAccelerationsCollapse()` si usan `collapse` en bucles anidados

2. NBodySimulator — integración temporal:

- `integrateEuler()`
- `integrateEuler(int sync_type)` con 0=atomic, 1=critical, 2=nowait (según la convención del curso; ver párrafo anterior)
- `integrateEuler(int sync_type, bool use_barrier)`

3. NBodySimulator — energía y métricas globales:

- `calculateEnergy()`
- `calculateEnergy(int method)` con 0=reduce, 1=atomic (sobre la parte que documenten, p.ej. $K + U$)
- `calculateEnergy(int method, bool use_private)`

4. NBodySimulator — reparto de trabajo sobre índices:

- `processBodies()`
- `processBodies(int task_type)` con 0=task, 1=parallel for (mismo esquema de partición en ambos casos)
- `processBodies(int task_type, bool use_single)`

5. Métodos dedicados a cláusulas puntuales:

- `simulatePhasesBarrier()`
- `parallelInitializationSingle()`
- `calculateMetricsFirstprivate()`
- `calculateFinalStateLastprivate()`

7.3. Benchmarks requeridos

1. Schedules: static, dynamic y guided; variar *chunk*; comentar balance de carga.
2. Sincronización: atomic, critical y reduce donde corresponda; discutir contención.
3. Datos: private vs shared; firstprivate vs lastprivate; patrones de acceso a memoria.
4. Sincronización avanzada: barrier vs nowait; task vs parallel for; single (y, si lo usan, master) en inicialización.
5. Escalabilidad: speedup y eficiencia vs número de hilos; contraste con Amdahl.
6. Fracción serial: medir porciones serial y paralelo con instrumentación explícita, comparar con la predicción de Amdahl y comentar *overhead* de OpenMP.

8. Parámetros de simulación

8.1. Configuración por defecto

- Número de cuerpos N : entre 500 y 2000 en corridas largas; subir a varios miles solo si la máquina y el tiempo de medición lo permiten.
- Pasos temporales: del orden de 500–2000; si el sistema diverge, descríbanlo y ajusten Δt o ε .
- Tiempo de simulación: indiquen en el README (y en el reporte cuando corresponda) la duración total, por ejemplo $t_{\text{fin}} = n_{\text{pasos}} \Delta t$ o el criterio de parada equivalente, para que los experimentos sean comparables.
- Paso Δt : pequeño respecto de la escala de velocidades; valores típicos de partida en unidades adimensionales suelen estar entre 10^{-3} y 10^{-2} , sujetos a calibración.
- Velocidades iniciales $\vec{v}_i(0)$: documenten cómo las fijan (reposo, órbita circular aproximada, muestreo del perfil inicial, etc.) y las unidades.
- Masas iniciales m_i : documenten rangos o reglas de asignación (por ejemplo uniforme en un intervalo, dos masas dominantes, perfil tipo Plummer) y declaren si hay masas nulas o degeneradas prohibidas.
- Constante G : puede fijarse en 1,0 en un sistema de unidades coherente declarado en el README.
- Suavizado ε : entre 10^{-2} y 10^{-1} en la misma unidad de longitud que las posiciones; eviten $\varepsilon = 0$.

8.2. Condiciones iniciales sugeridas

- Sistema binario con perturbación: dos masas grandes y una ligera, con semilla fija.
- Disco: posiciones en anillo o círculo, velocidades tangenciales aproximadas o en reposo.
- Opcional: perfil tipo Plummer proyectado a 2D, con referencia en el reporte.

9. Análisis y visualización

9.1. Gráficos requeridos

1. Speedup frente al número de hilos (varias variantes si aplica).
2. Eficiencia frente al número de hilos.
3. Tiempo frente al tamaño de *chunk* para distintos schedules.
4. Curva o puntos de Amdahl: predicción teórica vs mediciones.
5. Trayectorias o estado global: posiciones de un subconjunto de cuerpos en el tiempo, o evolución del centro de masas y del radio RMS.
6. Energía total $E = K + U$ en el tiempo; interpreten la deriva (Euler explícito y suavizado no conservan E de forma exacta).

9.2. Archivos de salida

- `benchmark_results.dat` — tablas de benchmarks
- `scaling_analysis.dat` — escalabilidad
- `performance_plots.png` — figuras de rendimiento
- `trajectories.dat` o `snapshots.dat` — posiciones (y opcionalmente velocidades) muestreadas
- `energy_timeseries.dat` — $K(t)$, $U(t)$, $E(t)$ u otras magnitudes globales

10. Pruebas automáticas

Incluyan pruebas unitarias y de integración, ejecutables con `make test` y en el pipeline de CI.

- Unitarias: fuerza o aceleración entre dos o tres cuerpos con geometría fija frente a valores analíticos; `zeroAccelerations`; masas no negativas.
- Acción–reacción entre pares: con posiciones (y masas) fijas, verificar que la fuerza que el cuerpo j ejerce sobre i sea opuesta a la que i ejerce sobre j , es decir $\vec{F}_{ij} = -\vec{F}_{ji}$ (equivalentemente $m_i \vec{a}_{i \leftarrow j} = -m_j \vec{a}_{j \leftarrow i}$ si aíslan la contribución del par), con tolerancia en coma flotante justificada.
- Integración: pocos pasos con N pequeño y semilla fija; comparación serial vs paralelo con tolerancia en coma flotante justificada; opcionalmente una referencia de un solo hilo.
- Regresión: al menos un caso que detecte errores graves (por ejemplo resultados insensibles a una masa no nula, o desviaciones grandes frente a referencia serial) o escrituras indebidas en aceleraciones.
- Cobertura: no hay meta numérica obligatoria, pero toda clase pública con lógica sustantiva debería tener al menos un test o una explicación en el README.

11. Instrucciones de compilación

11.1. Makefile

El Makefile debe incluir, como mínimo, lo siguiente:

```
1 CXX = g++
2 CXXFLAGS = -Wall -Wextra -O3 -fopenmp -std=c++17
3 LDFlags = -fopenmp
4
5 TARGET = nbody_2d
6 SOURCES = main.cpp Particle.cpp NBodySystem.cpp NBodySimulator.cpp Integrator.cpp
7 ↪ MetricsCalculator.cpp Benchmark.cpp Visualizer.cpp
8 HEADERS = Particle.h NBodySystem.h NBodySimulator.h Integrator.h MetricsCalculator.h
9 ↪ Benchmark.h Visualizer.h
10
11 $(TARGET): $(SOURCES) $(HEADERS)
12     $(CXX) $(CXXFLAGS) -o $(TARGET) $(SOURCES) $(LDFlags)
```

```

11
12 clean:
13     rm -f $(TARGET) *.o *.dat *.png
14
15 benchmark: $(TARGET)
16     ./${TARGET} -benchmark
17
18 analysis: $(TARGET)
19     ./${TARGET} -analysis
20
21 # Enlazar con GoogleTest/Catch2 según el proyecto; deberá ejecutarse en CI
22 test:
23     # Compilar fuentes de tests/ y dependencias; ejemplo:
24     # £(CXX) £(CXXFLAGS) -o run_tests tests/test_main.cpp NBodySystem.cpp Particle.cpp
25     ↪ £(LDFLAGS) -lgtest -lgtest_main
26     ./run_tests
27
28 .PHONY: clean benchmark analysis test

```

El objetivo `test` debe compilar y ejecutar la suite (GoogleTest, Catch2 u otro marco acordado). El Dockerfile debe permitir compilar, correr `make test` y una demostración breve del simulador, en línea con el syllabus del curso.

11.2. Compilación

```

1  # Compilar
2  make
3
4  # Ejecutar benchmarks
5  make benchmark
6
7  # Ejecutar análisis
8  make analysis
9
10 # Pruebas unitarias e integración
11 make test
12
13 # Limpiar
14 make clean

```

12. Criterios de evaluación

La nota pondera cinco aspectos. En cada ítem se valora que lo pedido en el enunciado esté implementado, medido y explicado con criterio, no solo que exista un archivo o un gráfico suelto.

12.1. Implementación (35 %)

- Correcta implementación de todas las cláusulas OpenMP en métodos de clase
- Diseño orientado a objetos con encapsulación apropiada
- Manejo de memoria y errores en C++
- Documentación del código con comentarios de clase

12.2. Pruebas, CI y contenedores (5 %)

- Suite `make test` significativa (unitarias + integración) y pipeline CI que la ejecute
- Dockerfile reproducible coherente con el README

12.3. Benchmarks (30 %)

- Implementación completa del sistema de benchmarks
- Comparaciones entre diferentes implementaciones
- Análisis de escalabilidad
- Cálculos correctos de métricas
- Implementación correcta de propagación de errores en speedup y eficiencia

12.4. Análisis y visualización (20 %)

- Gráficos de rendimiento
- Análisis de la ley de Amdahl
- Interpretación de resultados físicos
- Conclusiones sobre performance
- Presentación correcta de resultados con barras de error

12.5. Reporte técnico (10 %)

- Estructura y organización del reporte
- Claridad en la explicación del diseño
- Análisis crítico de los resultados
- Conclusiones bien fundamentadas
- Calidad de la presentación y formato

13. Entregables

La entrega oficial en **Google Classroom** son dos elementos: el **enlace (URL) al repositorio Git** del proyecto y el **PDF del reporte técnico**. Pueden ir en un mismo envío o en dos, según permita la plataforma. No se solicita archivo comprimido del código: el repositorio es la fuente de verdad del proyecto; el acceso del cuerpo docente y los requisitos de CI/tests se detallan en la sección *Fecha de entrega*.

Contenido mínimo del repositorio Git:

1. Código fuente completo (`.cpp`, `.h`) y carpeta `tests/`.
2. `Makefile` con objetivos al menos `all`, `clean` y `test`.
3. `Dockerfile` y configuración de CI (GitHub Actions, GitLab CI u otra).
4. `README.md` con compilación, pruebas, roles, **URL del repositorio** y cómo repetir los experimentos.
5. Tablas de benchmarks en archivos `.dat`.
6. Figuras de rendimiento en PNG (u otro formato vectorial/raster acordado con el curso).

Reporte técnico (PDF en Classroom): el PDF adjunto en la entrega debe tener hasta 10 páginas e incluir, como mínimo:

- Introducción: problema y objetivos del trabajo.
- Diseño: arquitectura de clases, reparto por rol y decisiones relevantes.
- Implementación: cómo mapearon las cláusulas OpenMP obligatorias mediante sobrecarga, citando métodos o fragmentos concretos.
- Resultados: speedup, eficiencia, Amdahl y propagación de errores donde corresponda.
- Análisis: comparación de schedules y cláusulas, interpretación física cuando aplique.
- Conclusiones: síntesis y limitaciones.
- Anexos: figuras extensas, tablas o código de apoyo.

14. Fecha de entrega

Publicación del enunciado: jueves 2 de abril de 2026.

Entrega: viernes 8 de mayo de 2026, 23:59 (coincide con la publicación del Laboratorio 2 en el programa del curso).

Formato:

- Plataforma: Google Classroom del curso.
- Enlace al repositorio Git: entreguen la URL del repositorio (visible en el texto de la entrega o como archivo/enlace adjunto) con el proyecto completo. El cuerpo docente debe tener acceso para revisar CI/CD y comprobar que los tests se ejecutan y pasan: repositorio *público*, o repositorio privado invitando al profesor y al ayudante como colaboradores o miembros con permisos suficientes (p.ej. rol de desarrollador o equivalente en la plataforma) para ver pipelines, registros de CI y resultados de `make test`. La misma URL debe figurar en el `README.md`.
- Reporte: PDF del reporte técnico (misma entrega en Classroom que el enlace al Git, o envío separado si Classroom lo permite).

15. Recursos adicionales

- Documentación OpenMP: <https://www.openmp.org/>
- GCC: <https://gcc.gnu.org/>
- Referencia C++: <https://en.cppreference.com/>
- Perfilado y depuración: gprof, Valgrind
- Visualización: Python/Matplotlib, Gnuplot
- Problema N-cuerpos: https://en.wikipedia.org/wiki/N-body_problem
- Modelo de Plummer / suavizado: https://en.wikipedia.org/wiki/Plummer_model

16. Consejos de implementación

- Partan con $N = 2$ o $N = 3$ en serial; recién después aumenten N y activen OpenMP.
- Un ε muy grande suaviza en exceso las fuerzas; uno muy pequeño acerca singularidades numéricas.
- Activen y prueben cada cláusula o variante de forma incremental.
- gdb, Valgrind y gprof son útiles para errores silenciosos y cuellos de botella.
- Repitan cada medición de benchmark varias veces antes de sacar conclusiones.
- Lleven un registro breve de decisiones y problemas encontrados; facilita escribir el reporte al final.

17. Recordatorios

- El proyecto debe compilar sin advertencias del compilador (`-Wall -Wextra`).
- Incluyan manejo razonable de errores (entradas inválidas, fallos de I/O, etc.).
- Los experimentos deben ser reproducibles desde el README y el repositorio.
- Documenten supuestos y límites del modelo o del hardware usado.
- El análisis de rendimiento debe apoyarse en números, no solo en impresiones cualitativas.
- Usen sobrecarga de funciones para evitar duplicar lógica entre variantes OpenMP.
- Cada cláusula obligatoria debe tener una ruta de ejecución verificable (test o benchmark), como se explicó en la sección de métodos.
- Los benchmarks requieren repeticiones suficientes para que promedios e incertidumbres tengan sentido.
- Mantengan encapsulación clara en las clases públicas del diseño.
- En el README o en el reporte, dejen explícitos Δt , ε , unidades y el criterio de tolerancia en comparaciones de coma flotante.

- Los métodos “simples” deben delegar en las versiones parametrizadas cuando el diseño lo permita, para no bifurcar la lógica en dos sitios distintos.